



## 第十届

# 全国大学生集成电路创新创业大赛

报告类型：设计报告

参赛杯赛：“竞业达”企业命题

作品名称：基于分支预测与系统控制机制的  
RISC-V 流水线处理器

队伍编号：CICC1000288

团队名称：卡了电竞

# 目录

|                                 |            |
|---------------------------------|------------|
| <b>快速预览简介</b>                   | <b>III</b> |
| <b>RISC-V CPU 设计报告</b>          | <b>1</b>   |
| <b>1 项目概述</b>                   | <b>1</b>   |
| 1.1 项目背景 . . . . .              | 1          |
| 1.2 设计目标 . . . . .              | 1          |
| 1.3 设计平台说明 . . . . .            | 2          |
| <b>2 RISC-V CPU 架构设计</b>        | <b>2</b>   |
| 2.1 RISC-V 指令集的支持情况 . . . . .   | 2          |
| 2.2 RISC-V CPU 的整体架构图 . . . . . | 3          |
| 2.3 数据通路各模块的设计 . . . . .        | 4          |
| 2.3.1 CPU 数据通路整体说明 . . . . .    | 4          |
| <b>3 前端预测与控制通路设计</b>            | <b>5</b>   |
| 3.1 PC 更新与优先级 . . . . .         | 5          |
| 3.2 gshare 方向预测器 . . . . .      | 5          |
| 3.3 BTB、返回地址栈与预测目标 . . . . .    | 5          |
| 3.4 解析、训练与错误恢复 . . . . .        | 6          |
| <b>4 译码、执行与长周期运算设计</b>          | <b>6</b>   |
| 4.1 译码与立即数生成 . . . . .          | 6          |
| 4.2 整数执行单元 . . . . .            | 6          |
| 4.3 乘法与除法 . . . . .             | 7          |
| <b>5 流水线冒险处理机制</b>              | <b>7</b>   |
| 5.1 数据前递网络 . . . . .            | 7          |
| 5.2 load-use 停顿 . . . . .       | 8          |
| 5.3 停顿、冲刷与有效位 . . . . .         | 8          |
| <b>6 访存通路 &amp; 双端口 BRAM 设计</b> | <b>9</b>   |
| 6.1 载入存储请求生成 . . . . .          | 9          |
| 6.2 双端口存储结构 . . . . .           | 9          |

|           |                             |           |
|-----------|-----------------------------|-----------|
| 6.3       | 写后读前递 . . . . .             | 9         |
| <b>7</b>  | <b>CSR 与机器模式控制设计</b>        | <b>10</b> |
| 7.1       | CSR 文件与访问语义 . . . . .       | 10        |
| 7.2       | ecall 进入流程 . . . . .        | 10        |
| 7.3       | mret 返回流程 . . . . .         | 11        |
| <b>8</b>  | <b>功能仿真与验证</b>              | <b>11</b> |
| 8.1       | 验证方法 . . . . .              | 11        |
| 8.2       | 双端口 BRAM 功能仿真 . . . . .     | 12        |
| 8.3       | BPU 功能仿真 . . . . .          | 13        |
| 8.4       | ecall 与 mret 功能仿真 . . . . . | 14        |
| <b>9</b>  | <b>系统集成与设计特点</b>            | <b>14</b> |
| 9.1       | 模块化组织 . . . . .             | 14        |
| 9.2       | 面向同步存储器的流水设计 . . . . .      | 15        |
| 9.3       | 当前功能边界 . . . . .            | 15        |
| <b>10</b> | <b>总结与展望</b>                | <b>15</b> |
| <b>A</b>  | <b>主要 RTL 文件结构与职责对应表</b>    | <b>16</b> |

## 快速预览简介

本作品面向“竞业达”企业命题，设计并实现了一款基于 RISC-V 的 32 位流水线 CPU。处理器以顺序发射、顺序完成为基本执行方式，主数据通路由 IF、ID、EX、MEM1、MEM2、WB 六个阶段组成，并在控制转移路径中增加 EX2 结果寄存边界。在保持功能正确的基础上，设计加入了寄存器前递、load-use 停顿、预测错误恢复、同步访存响应、双端口 BRAM 写后读前递以及机器模式系统控制等机制。

### 关键技术

- **流水线数据通路**：完成取指、译码、执行、两级访存与写回的连接，使用有效位控制指令推进、气泡插入和错误路径冲刷。
- **数据冒险处理**：在 ID 阶段预计算前递选择，在 EX 阶段从 EX/MEM、MEM2 与 MEM/WB 三个位置取得最新操作数；紧邻 load-use 相关只暂停一个周期。
- **分支预测与恢复**：使用 gshare、两位饱和计数器表和 BTB 预测控制流，在 EX2 同时核对方向和目标地址，错误时统一重定向 PC。
- **同步存储系统**：使用 MEM1/MEM2 响应边界对接同步读存储器；系统层通过真双端口 BRAM 与单项写缓冲处理读写并行和写后读相关。
- **系统控制**：实现机器模式 CSR、ecall 入口状态保存和 mret 返回流程，保证系统控制转移与普通跳转共用冲刷机制。

### 主要功能

- 支持常用 RV32I 整数、分支、跳转、载入和存储指令，以及 RV32M 乘、除、余数运算；
- 支持 CSRRW/CSRRS/CSRRC 及其立即数形式，并实现 mstatus、mtvec、mepc、mcause 与 mscratch；
- 通过双端口 BRAM、BPU、ecall/mret 三组功能仿真观察关键状态和控制信号。

### 作品亮点

- **结构清晰**：阶段模块、流水寄存器、运算单元和预测控制模块分层组织，便于独立验证与后续扩展；
- **控制完整**：对数据相关、长周期运算、预测错误和系统调用均设置了有效位、停顿或重定向控制，避免错误路径产生副作用；
- **报告可追溯**：正文按模块、信号和控制流程说明实现逻辑，仿真部分直接对应双端口 BRAM、BPU 与 ecall/mret 三项验证内容。

# RISC-V CPU 设计报告

## 1 项目概述

### 1.1 项目背景

RISC-V 指令集采用开放、模块化的组织方式，适合用于处理器体系结构教学与 FPGA 原型验证 [1]。与单周期实现相比，流水线处理器把取指、译码、执行、访存和写回分散到不同周期，使多条指令能够重叠执行；与此同时，数据相关、控制相关、同步存储器延迟以及长周期运算也成为必须显式处理的问题。

本项目结合“竞业达”企业命题的处理器设计场景，围绕可综合、可仿真、可扩展的 RISC-V CPU 展开实践。设计不仅需要完成指令功能，更需要构建一条能够连接 FPGA 存储器和外设环境的完整执行通路：正常指令结果应按程序顺序写回，相关指令必须取得最新操作数，错误路径不能产生寄存器、存储器或 CSR 副作用，系统调用进入处理程序后还应能够正确返回。

### 1.2 设计目标

本项目的总体目标是使用 Verilog HDL 设计并实现一款结构清晰、功能正确、能够在仿真环境中观察关键流水线状态的 32 位 RISC-V 处理器。具体设计目标如下：

1. **搭建完整流水线数据通路。**实现 IF、ID、EX、MEM1、MEM2、WB 六个主阶段及 EX2 控制结果边界，完成取指、译码、执行、访存和写回流程。
2. **实现整数、访存和长周期运算。**支持常用 RV32I 指令、RV32M 乘除法，并实现字节、半字、字访问及相应的数据格式化。
3. **处理流水线冒险。**通过前递、load-use 暂停、长周期运算保持与重定向冲刷，解决数据相关和控制相关问题。
4. **实现动态控制流机制。**构建由 gshare、PHT、BTB 和返回地址栈组成的前端预测结构，并在 EX2 对实际方向与目标进行核对和恢复。
5. **完成系统控制与存储验证。**实现 CSR、ecall、mret 和双端口 BRAM 写后读前递，并以三组波形验证关键功能。

## 1.3 设计平台说明

表 1.1 设计平台说明表

| 项目    | 内容                                         |
|-------|--------------------------------------------|
| 开发平台  | Xilinx Vivado 2023.2                       |
| 设计语言  | Verilog HDL；系统集成与仿真接口使用 SystemVerilog      |
| 目标架构  | 面向 RV32I 基本整数功能, 扩展 RV32M 乘除法与机器模式 CSR 控制  |
| 数据位宽  | 32 位指令、32 位数据、32 个通用寄存器                    |
| 主流流水线 | IF、ID、EX、MEM1、MEM2、WB 六级, 另设 EX2 控制结果边界    |
| 主要模块  | IFU、IDU、EXU、LSU、WBU、Regfile、BPU、CSR 与流水寄存器 |
| 存储系统  | 16 KiB 指令存储空间；256 KiB 双端口数据 BRAM 与存储器映射外设  |
| 验证方式  | 双端口 BRAM、BPU、ecall/mret 功能仿真与关键信号观察        |

## 2 RISC-V CPU 架构设计

### 2.1 RISC-V 指令集的支持情况

译码模块 KLDJ\_idu 直接解析 opcode、funct3 和 funct7, 并并行生成操作类型、寄存器地址、立即数和控制信号。其主要指令功能如表 2.1 所示。

表 2.1 处理器主要指令功能

| 类别    | 指令                                        | 功能说明                 |
|-------|-------------------------------------------|----------------------|
| 立即数运算 | ADDI, SLTI, SLTIU, XORI, ORI, ANDI        | 立即数算术、比较和逻辑运算        |
| 移位运算  | SLLI, SRLI, SRAI, SLL, SRL, SRA           | 逻辑左移、逻辑右移和算术右移       |
| 寄存器运算 | ADD, SUB, SLT, SLTU, XOR, OR, AND         | 双寄存器算术、比较和逻辑运算       |
| 高位立即数 | LUI, AUIPC                                | 生成高位立即数或计算 PC 相对地址   |
| 控制转移  | JAL, JALR, BEQ, BNE, BLT, BGE, BLTU, BGEU | 无条件跳转、间接跳转和六类条件分支    |
| 载入    | LB, LH, LW, LBU, LHU                      | 字节、半字和字读取, 并进行符号或零扩展 |

| 类别     | 指令                                             | 功能说明                 |
|--------|------------------------------------------------|----------------------|
| 存储     | SB, SH, SW                                     | 通过四位字节使能完成字节、半字和字写入  |
| 乘除法    | MUL, MULH, MULHSU, MULHU, DIV, DIVU, REM, REMU | 有符号/无符号乘法高低位、除法和余数运算 |
| CSR 访问 | CSRRW, CSRRS, CSRRC, CSRRWI, CSRRSI, CSRRCI    | CSR 交换、置位、清位及其立即数形式  |
| 系统控制   | ECALL, MRET                                    | 进入机器模式处理入口并从处理程序返回   |

## 2.2 RISC-V CPU 的整体架构图

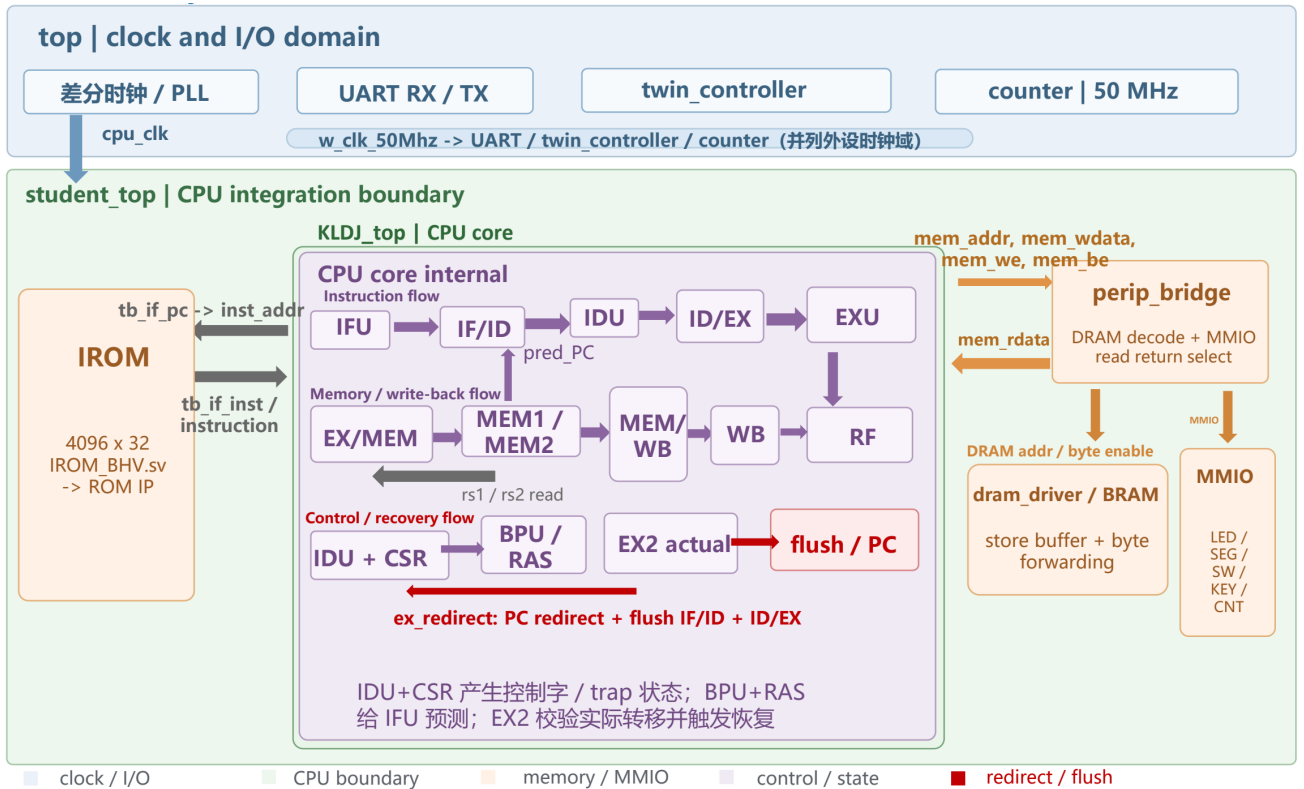


图 2.1 RISC-V 流水线 CPU、存储器与外设集成架构

图 2.1 按时钟与 I/O 域、student\_top 集成边界和 KLDJ\_top 核心三个层次展开。顶层处理差分时钟/PLL、UART、twin\_controller 和计数器等板级接口；student\_top 在 CPU 与 IROM、perip\_bridge、dram\_driver/BRAM 和 MMIO 之间完成连接。核心

内部的指令流依次经过 IFU、IF/ID、IDU、ID/EX 与 EXU，访存/写回流经 EX/MEM、MEM1/MEM2、MEM/WB、WB 和寄存器堆；IDU+CSR、BPU/RAS 与 EX2 构成独立的控制恢复通路。图中的红色 `ex_redirect` 明确给出预测错误或系统控制转移时对 PC 的重定向，以及对 IF/ID、ID/EX 的冲刷。

核心模块 `KLDJ_top` 的外部接口可分为取指和数据访存两组。指令由 `tb_if_inst` 输入，当前取指 PC 通过 `tb_if_pc` 输出；数据侧输出 `mem_addr`、`mem_wdata`、`mem_we` 和四位 `mem_be`，同步读响应从 `mem_rdata` 返回。

系统顶层将 PC 的 [13:2] 位连接到 4096 深度的指令存储器，因此指令空间容量为 16 KiB。数据请求先进入 `perip_bridge`：0x8010\_0000 至 0x8013\_FFFF 映射到 256 KiB 数据 BRAM；0x8020\_0000 区域映射到拨码开关、按键、数码管、LED 和计数器等外设。桥接模块把读地址类型寄存一拍，使同步 DRAM 响应和 MMIO 响应在同一时序边界返回核心。

## 2.3 数据通路各模块的设计

### 2.3.1 CPU 数据通路整体说明

主数据通路由六个阶段构成，阶段职责见表 2.2。其中 EX2 只保存分支、跳转和系统控制指令的判定结果，和普通运算的 EX/MEM 数据通路并行，不应作为所有指令都经过的第七级。`pipe_wb_commit` 则是一个提交观察寄存器；寄存器堆的实际写入由 MEM/WB 状态驱动，因此该观察点也不改变体系结构写回级数。

表 2.2 主流水线各阶段职责

| 阶段          | 主要模块                                                        | 功能                       |
|-------------|-------------------------------------------------------------|--------------------------|
| <b>IF</b>   | <code>KLDJ_ifu</code> , <code>bpu</code>                    | 保存 PC，接收指令，计算顺序 PC 或预测目标 |
| <b>ID</b>   | <code>KLDJ_idu</code> ,<br><code>KLDJ_regfile</code>        | 译码、立即数生成、寄存器读取、前递选择预计算   |
| <b>EX</b>   | <code>KLDJ_exu</code> ,<br><code>ex_forward</code>          | ALU、比较、乘除法、地址生成和操作数前递    |
| <b>MEM1</b> | <code>pipe_ex_mem</code>                                    | 保存访存元数据，等待同步存储器响应        |
| <b>MEM2</b> | <code>pipe_mem1_mem2</code> ,<br><code>mem_stage_top</code> | 对载入数据选字节/半字并扩展，形成可前递结果   |
| <b>WB</b>   | <code>pipe_mem_wb</code> ,<br><code>KLDJ_wbu</code>         | 锁存写回数据并写入通用寄存器堆          |

结合图 2.1，寄存器堆的读数据沿指令流送入 IDU，写回结果则由 WB 返回 RF；EX/MEM、MEM1/MEM2 和 MEM/WB 提供三路前递来源。BPU/RAS 在 IF 阶段给出预测 PC，EX2 接收实际转移结果；发生错误时，`flush` / PC 单元把 `ex_redirect` 返回 IF，并清除更年轻的流水线状态。数据存储器请求在 EX 阶段发出，响应经过 MEM1/MEM2



边界寄存后再参与前递和写回。

### 3 前端预测与控制通路设计

#### 3.1 PC 更新与优先级

KLDJ\_ifu 内部使用 32 位 pc\_reg 保存当前 PC，顺序地址为

$$\text{snpc} = \text{PC} + 4. \quad (1)$$

若前端预测转移，则下一 PC 取预测目标，否则取 snpc。时序控制的优先级为：复位、执行级重定向、前端保持、正常预测/顺序更新。重定向高于停顿，保证即使流水线因数据相关或长周期运算保持，已经解析出的错误路径仍能立即触发恢复。

IF/ID 寄存器除指令和 PC 外，还保存预测方向、预测目标与 PHT 索引。预测索引随指令进入 EX2，更新时使用取指当时的索引，而不是用已经变化的 GHR 重新计算，从而避免历史状态改变导致训练错误表项。

#### 3.2 gshare 方向预测器

BPU 的索引宽度为 6，因此模式历史表 PHT 包含 64 个两位饱和计数器，全局历史寄存器 GHR 也为 6 位。查表索引为

$$i_{\text{PHT}} = \text{PC}[7:2] \oplus \text{GHR}[5:0]. \quad (2)$$

计数器最高位为 1 时预测跳转。未训练表项通过独立有效位识别，其默认值为 2'b01，即弱不跳转。实际结果为跳转时计数器加一，为不跳转时减一，并在 00 和 11 处饱和。每次有效的条件分支或直接跳转解析后，实际方向移入 GHR。

PHT 数据阵列不整体复位，只清除有效位。这一做法既能保证复位后预测状态确定，又便于综合工具将数据阵列映射为分布式存储器，避免为每个表项建立复位逻辑。

#### 3.3 BTB、返回地址栈与预测目标

分支目标缓冲表 BTB 同样包含 64 项，以 PC[7:2] 直接索引，每项保存高位标签和 32 位目标地址。只有 BTB 有效、标签匹配且 PHT 预测跳转时，BPU 才同时给出有效方向与目标。表项更新被寄存一拍，实际跳转时写入目标地址；不跳转结果只训练 PHT，不破坏原有 BTB 目标。

设计还包含深度为 8 的循环返回地址栈。以 x1 或 x5 为链接寄存器的 JAL/JALR 可

压入 PC+4，规范形式 jalr x0, 0(x1/x5) 可弹栈。栈满时覆盖最旧项，空栈弹出无效，同周期弹出和压入则替换栈顶。板级默认配置保留栈的更新逻辑，但不让异步取指结果经返回地址预测直接反馈到 PC；该功能可通过顶层参数在适合的配置中启用。

### 3.4 解析、训练与错误恢复

条件分支、JAL 和 JALR 的实际方向与目标在 EX 形成，并经 EX2 寄存后交给 ex\_bpu\_ctrl。错误判断同时覆盖方向和目标：

$$\text{direction\_miss} = p_t \oplus a_t, \quad (3)$$

$$\text{target\_miss} = p_t \wedge a_t \wedge (P_{\text{target}} \neq A_{\text{target}}), \quad (4)$$

$$\text{redirect} = \text{valid} \wedge (\text{direction\_miss} \vee \text{target\_miss}), \quad (5)$$

其中  $p_t$  和  $a_t$  分别表示预测与实际是否转移。仅检查方向会漏掉“方向正确但目标错误”的间接跳转，因此目标比较是保证正确性的必要条件。

发生重定向时，IF/ID 和 ID/EX 的有效位被清零，当前更年轻的 EX 指令也通过 ex\_current\_valid 屏蔽。存储写使能、CSR 写使能和后续流水寄存器都由该有效信号保护，所以错误路径不会留下体系结构副作用。

## 4 译码、执行与长周期运算设计

### 4.1 译码与立即数生成

译码级并行生成 I、S、B、U 和 J 五种立即数，所有需要符号扩展的格式均从指令位 31 复制符号位。控制逻辑采用并行 case 选择，不建立跨全部指令的深层优先链。译码结果包括 18 位内部操作码、10 位 ALU 控制、源/目的寄存器地址、读写使能、四路数据操作数以及四位访存控制。

寄存器堆具有两个组合读端口和一个同步写端口，x0 始终保持为零。若同一周期写回地址与读地址相同，读端口直接返回写回数据，形成寄存器堆内部的 write-through 旁路。这一机制覆盖 WB 到 ID 的相关，减少额外比较层级。

### 4.2 整数执行单元

KLDJ\_alu 将加减比较、逻辑与/或/异或、移位和小于比较拆分为独立子模块，再由 one-hot 风格的控制位选择结果。加法器同时输出等于、不等、小于和大于等于标志，条

件分支直接复用这些标志。分支目标、直接跳转目标和访存地址分别为

$$A_{branch} = PC + imm_B, \quad (6)$$

$$A_{jal} = PC + imm_J, \quad (7)$$

$$A_{mem} = rs1 + imm. \quad (8)$$

JALR 目标为  $(rs1 + imm) \& 0xFFFF\_FFFE$ ，即按指令语义清除最低位；JAL/JALR 写回值均为  $PC+4$ 。

地址生成单独使用 `ls_imm`，不经过存储数据的 `rs2` 前递多路器。这样，存储数据相关只影响待写数据，不会被误接入地址生成组合路径。

### 4.3 乘法与除法

四类乘法由 `mul_ip_wrapper` 统一处理。操作数先扩展为 33 位，根据 `MULH`、`MULHSU` 和 `MULHU` 的符号组合选择符号扩展或零扩展，再从 66 位乘积中选择低 32 位或原始 32 位乘法结果的高 32 位。RTL 包装器通过 `start/busy/done` 协议管理固定延迟乘法 IP，默认等待计数为 4 个周期。

除法包装器把操作划分为 `IDLE`、`LAUNCH`、`WAIT_RESULT` 和 `DONE` 四种状态。带符号运算先求绝对值，在 IP 返回商和余数后恢复符号。设计对两个边界条件作了显式处理：

- 除数为零时，`DIV/DIVU` 返回全 1，`REM/REMU` 返回被除数；
- $0x8000\_0000 / 0xFFFF\_FFFF$  的有符号溢出按指令语义返回，避免进入 IP。

乘除法未完成时，`mul_stall` 或 `div_stall` 拉高。此时 `IF/ID` 和 `ID/EX` 保持原内容，`EX/MEM` 注入无效项，直至 `done` 有效后长周期指令才向后推进，因此同一条指令不会被重复发射。

## 5 流水线冒险处理机制

### 5.1 数据前递网络

数据前递由 `pipe_id_ex` 和 `ex_forward` 协同完成。ID 阶段比较当前指令源寄存器与下游目的寄存器，并把两位前递选择随指令寄存。下一周期 `EX` 只需通过四选一多路器选择操作数，避免在 `EX` 的关键组合路径中再多做多组 5 位地址比较。

表 5.1 EX 阶段前递来源及优先级

| 选择编码 | 数据来源        | 适用情况             |
|------|-------------|------------------|
| 00   | ID/EX 原始操作数 | 无相关或寄存器堆已提供最新值   |
| 01   | EX/MEM 运算结果 | 最近一条非载入生产者，优先级最高 |
| 10   | MEM2 前递值    | 更早的运算结果或已格式化载入结果 |
| 11   | MEM/WB 写回值  | 尚未由寄存器堆读出的写回数据   |

rs1 和 rs2 各有独立选择器。存储指令的写数据使用与 rs2 相同的选择结果，但未前递基值取自专门保存的 data3，因此“计算地址”和“准备写数据”保持逻辑分离。

## 5.2 load-use 停顿

载入数据经过同步存储器后才能得到。如果 IF/ID 指令读取的源寄存器等于当前 ID/EX 载入指令的目的寄存器，则产生 load-use 相关：

$$\text{stall}_{load} = V_L \wedge (rd_L \neq 0) \wedge V_C \wedge [(R_1 \wedge rs1_C = rd_L) \vee (R_2 \wedge rs2_C = rd_L)]. \quad (9)$$

停顿期间 PC 与 IF/ID 保持，ID/EX 注入一个无效项。由于存储响应在进入 MEM2 前已经完成字节/半字选择和符号扩展，下一周期可从 mem2\_forward\_data 直接前递到 EX，因此紧邻 load-use 相关只需一个气泡，而不必再等待到 WB。

## 5.3 停顿、冲刷与有效位

各级使用 valid 位区分真实指令和气泡。控制优先级可概括为：重定向冲刷高于长周期保持，长周期保持高于正常推进；load-use 在长周期运算结束后再注入气泡。该优先级避免乘除法执行过程中 ID/EX 内容被相关控制意外清除。

对体系结构副作用的保护不是只在前端清除指令，而是在每个副作用边界使用有效位：寄存器写使能与 MEM/WB 有效状态绑定，存储写使能与 ex\_current\_valid 绑定，CSR 写入、ecall 状态保存也使用同一有效条件。这使流水线冲刷具有端到端的一致语义。

## 6 访存通路与双端口 BRAM 设计

### 6.1 载入存储请求生成

EX 阶段计算有效地址，并由 `ex_mem_req_ctrl` 产生字节写使能。设地址低两位为  $a_1a_0$ ，请求大小为 byte、half 或 word，则

$$BE = \begin{cases} 0001 \ll a_1a_0, & \text{byte,} \\ 0011 \ll \{a_1, 0\}, & \text{half,} \\ 1111, & \text{word.} \end{cases} \quad (10)$$

字节存储把低 8 位复制到四个字节位置，半字存储把低 16 位复制两次，再由 BE 选择实际写入的字节。存储写使能只在指令有效、EX 未停顿且操作确为存储时拉高；无效或错误路径请求最多造成无副作用的读，不能写坏存储器。

载入响应在 MEM1/MEM2 边界前格式化。字节通过地址低两位从 32 位读数据中四选一，半字通过地址位 1 二选一，再根据控制位进行符号扩展或零扩展；字载入直接使用完整 32 位数据。格式化结果被寄存后同时服务于写回和 MEM2 前递，保证两条路径看到完全相同的体系结构值。

### 6.2 双端口存储结构

系统集成层的 `dram_driver` 使用真双端口 BRAM。端口 A 专用于写，写地址、数据和字节使能先寄存；端口 B 专用于读，组合地址进入 BRAM，读数据在时钟边沿后输出。读写分端口后，处理器可以在一个请求写入尚未完全离开存储器接口时继续给出读地址，也避免读写控制共享一个端口造成的结构冲突。

BRAM 容量为  $65536 \times 32$  bit，共 256 KiB，具有四位逐字节写使能。仿真模型中端口 A 采用 write-first 行为，端口 B 采用 read-first 行为；综合实现则由块存储器 IP 提供等价接口 [3]。

### 6.3 写后读前递

注册写路径改善了接口分级，但会产生新的 RAW 风险：存储指令刚把写请求送入寄存器，紧随其后的载入可能先从 BRAM 读到旧值。为此，`dram_driver` 设置一个单项写缓冲，保存最近一次写地址、写数据和四位字节使能，并用两位有效移位寄存器覆盖待写请求的有效窗口。

当读地址与缓冲地址相同时，前递判定、逐字节选择和待写数据再寄存一拍，以

便与 BRAM 端口 B 的同步读输出对齐。最终每个字节独立选择：

$$D_{out}[8i+7:8i] = \begin{cases} D_{buf}[8i+7:8i], & \text{fwd\_byte\_en}[i] = 1, \\ D_{bram}[8i+7:8i], & \text{其他情况}, \end{cases} \quad (11)$$

其中  $i = 0, 1, 2, 3$ 。逐字节合并同时支持 SB、SH 后的整字读取：被本次存储覆盖的字节来自缓冲，其余字节仍来自 BRAM。

## 7 CSR 与机器模式控制设计

### 7.1 CSR 文件与访问语义

KLDJ\_csr 实现表 7.1 所示的五个 32 位机器模式寄存器。读端口为组合逻辑，写入在时钟上升沿完成，因此 CSR 读改写指令在同一周期读到旧值并将其写回 rd，同时把计算后的新值写入 CSR，符合交换、置位和清位操作的基本语义 [2]。

表 7.1 已实现的机器模式 CSR

| 寄存器      | 地址    | 作用                      |
|----------|-------|-------------------------|
| mstatus  | 0x300 | 保存机器模式全局使能、前态使能和前一特权级   |
| mtvec    | 0x305 | 保存系统控制入口基地址             |
| mscratch | 0x340 | 供处理程序暂存软件上下文            |
| mepc     | 0x341 | 保存触发系统调用的指令 PC，也是返回目标来源 |
| mcause   | 0x342 | 保存原因码；机器模式环境调用写入 11     |

### 7.2 ecall 进入流程

译码级识别 SYSTEM 主操作码、funct3=000 且相应字段为零的 ecall。有效指令在 EX 被识别后，CSR 模块执行以下状态更新：

1. mepc 保存当前 ecall 指令的 PC；
2. mcause 写为 0x0000\_000B；
3. mstatus.MPIE 保存原 MIE，MIE 清零，MPP 置为机器模式；
4. EX2 控制单元把正确 PC 选为 mtvec，若此前未预测该转移则重定向前端；
5. IF/ID、ID/EX 与更年轻 EX 指令被冲刷，处理器从入口地址继续取指。

本实现将 mepc 保存为 ecall 自身地址，而不是 PC+4。因此，若处理程序希望返回到系统调用之后，软件应在执行 mret 前把 mepc 增加 4；这一点也是功能仿真中需要核对的关键行为。

### 7.3 mret 返回流程

mret 在 EX 读取当前 mepc 作为跳转目标。CSR 模块同时执行  $MIE \leftarrow MPIE$ 、 $MPIE \leftarrow 1$  和  $MPP \leftarrow 00$ 。其控制转移结果经 EX2 与原预测比较，必要时重定向到 mepc。和普通分支恢复相同，重定向时错误路径的存储写入与 CSR 写入均被有效位屏蔽。

## 8 功能仿真与验证

### 8.1 验证方法

功能验证采用定向激励与内部关键信号观察相结合的方法。三组仿真分别覆盖存储器接口、前端预测和系统控制；不仅检查最终数据，还检查请求、状态更新和重定向的周期关系。表 8.1 给出建议的判定要点。

表 8.1 三组功能仿真的验证目标

| 仿真对象       | 主要观察信号                                          | 通过条件                                               |
|------------|-------------------------------------------------|----------------------------------------------------|
| 双端口 BRAM   | 写使能、地址、字节使能、写数据、读地址、缓冲有效、读数据                    | 全字与局部写入正确；紧邻同址读取取得合并后的新值；异址读取不误前递                  |
| BPU        | lookup PC、PHT 索引、预测方向/目标、更新方向/目标、BTB 命中、GHR、重定向 | 冷启动预测确定；训练后计数器饱和变化；BTB 目标命中；方向或目标错误均触发恢复           |
| ecall/mret | 当前 PC、mepc、mcause、mtvec、mstatus、重定向与目标 PC       | ecall 跳到 mtvec 并保存状态；软件调整 mepc 后，mret 返回正确地址并恢复状态位 |

## 8.2 双端口 BRAM 功能仿真

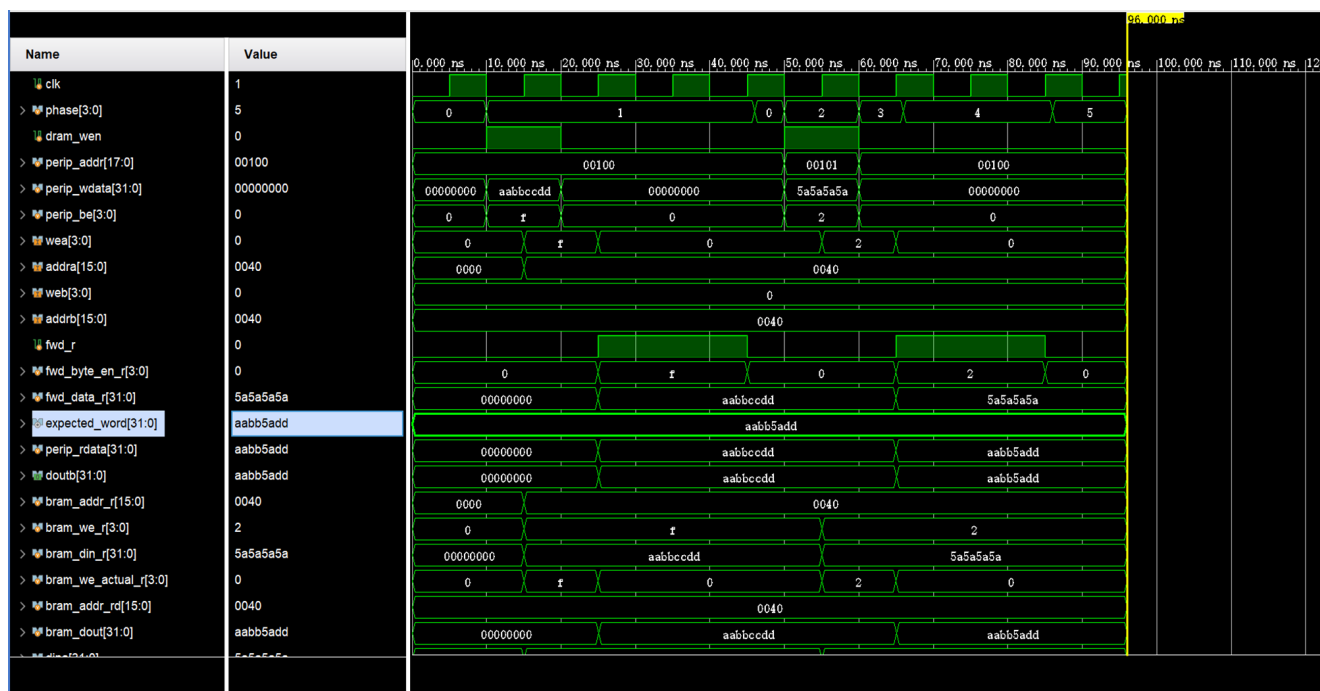


图 8.1 双端口 BRAM 全字写入、字节更新与写后读前递仿真

图 8.1 验证同一字地址 0x0040 的定向读写。全字写入阶段中，perip\_wdata 为 0xAABBCCDD，写使能为 4'b1111。经过同步读响应边界，expected\_rdata、perip\_rdata 和 doutb 均得到 0xAABBCCDD。随后在约 60 ns 对字节地址 0x00101 发起 4'b0010 的局部写入，写数据为 0x5A5A5A5A。

前递字节使能 fwd\_byte\_en\_r 同步变为 4'b0010，仅覆盖第二个字节；约 80 ns 后三路读数据一致为 0xAABB5ADD。这表明注册写路径下的同址读能够取得逐字节合并后的新值，而非 BRAM 的旧值。



### 8.3 BPU 功能仿真

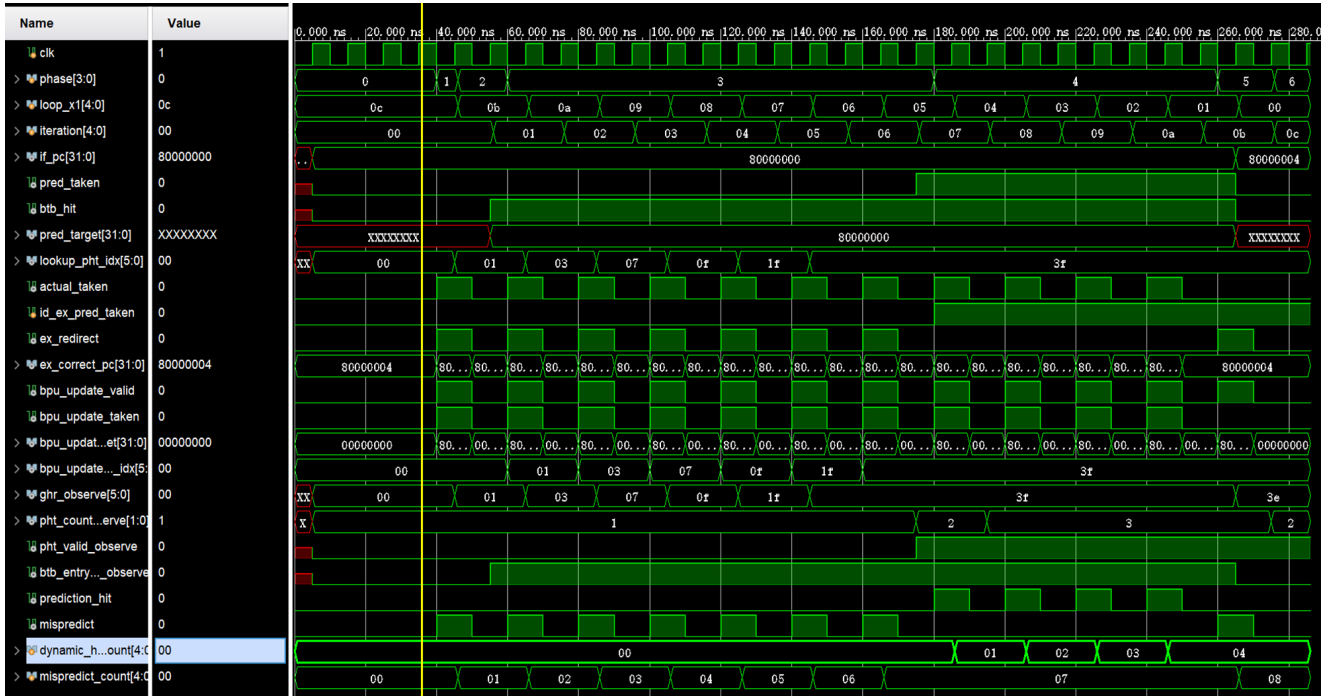


图 8.2 gshare、PHT 与 BTB 的训练、稳定预测和恢复仿真

图 8.2 以 0x80000000 处的循环分支验证预测器的状态演化。冷启动时 PHT 计数器为弱不跳转状态 01，连续的实际跳转使 ghr\_observe 依次从 00 演化为 01、03、07、0F、1F 和 3F，对应的查找/更新索引也按 gshare 规则变化。BTB 建立目标 0x80000000 后，PHT 计数器依次达到 10 和 11；约 160 ns 起 pred\_taken 有效，后续连续跳转不再触发 ex\_redirect。末次实际结果改为不跳转时，取指 PC 转为 0x80000004，GHR 更新为 3E，PHT 计数器由 11 回退为 10，并产生一次恢复。波形因此同时验证了冷启动训练、BTB 命中、稳态预测和方向失配恢复。

## 8.4 ecall 与 mret 功能仿真

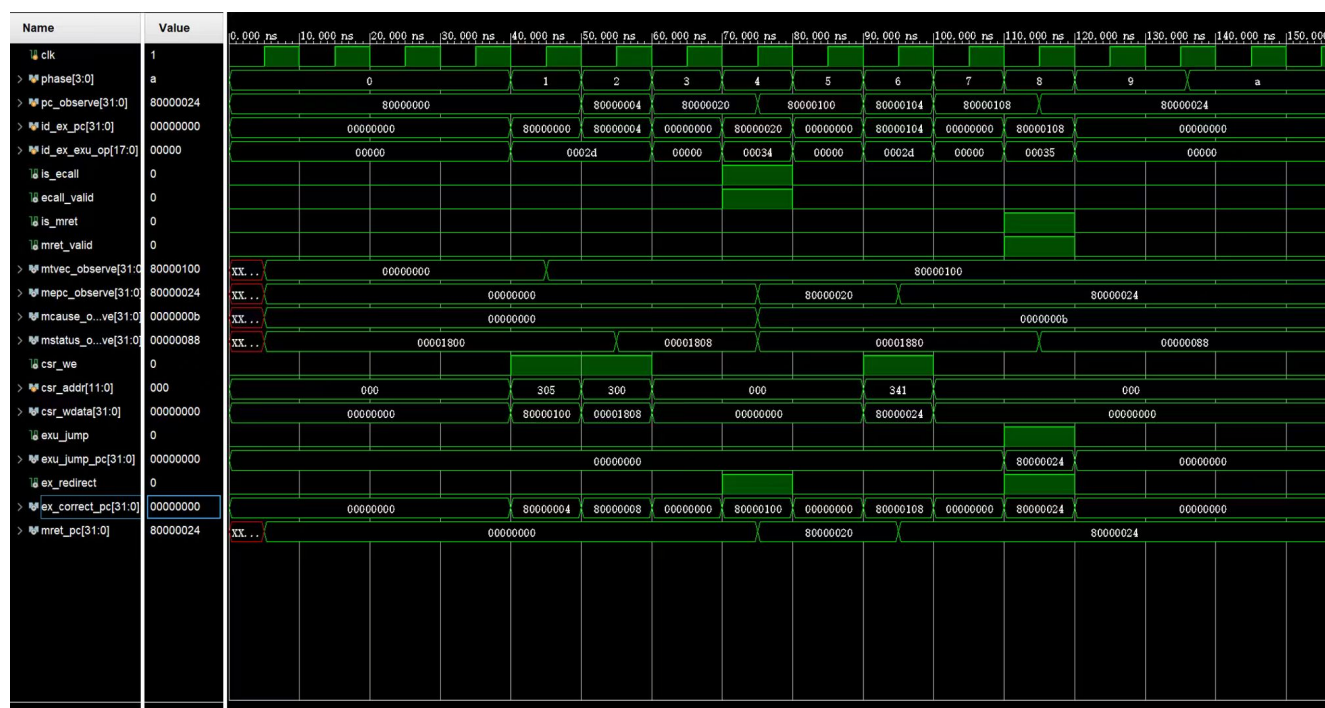


图 8.3 ecall 进入、CSR 状态更新与 mret 返回仿真

图 8.3 展示了完整的机器模式控制闭环。首先，测试程序通过 CSR 地址 0x305 将 `mtvec` 配置为 0x80000100，并通过地址 0x300 将 `mstatus` 置为 0x00001808。约 60 ns 时 `is_ecall` 和 `ecall_valid` 有效，`mepc` 捕获 0x80000020，`mcause` 更新为 0x0000000B，`mstatus` 变为 0x00001880，且 `ex_redirect` 将 PC 重定向到 0x80000100。处理程序随后经 CSR 地址 0x341 把 `mepc` 改写为 0x80000024。约 100 ns 时 `is_mret` 和 `mret_valid` 有效，`mret_pc` 与 `ex_jump_pc` 均为 0x80000024，重定向后的取指 PC 也稳定为该地址，同时 `mstatus` 恢复为 0x00000088。由此验证了 `ecall` 状态保存、软件调整返回地址和 `mret` 恢复执行的完整时序。

## 9 系统集成与设计特点

### 9.1 模块化组织

处理器将功能单元、流水寄存器和控制单元分目录组织。阶段模块负责体系结构功能，`pipe` 目录负责寄存边界、前递、预测和请求控制，`alu` 目录负责可复用运算单元。顶层主要完成数据连线 and 全局优先级组合，便于分别验证 BPU、乘除法包装器、存储驱动和完整处理器。

## 9.2 面向同步存储器的流水设计

本设计没有假定数据存储器组合读，而是显式设置 MEM1/MEM2 响应边界。载入数据先在寄存器前完成选字节和扩展，再以寄存值参与旁路。这种组织方式使 CPU 核心接口能够连接 FPGA BRAM 与带寄存响应的外设桥，同时避免把存储器实时输出直接接入 EX 多路器。

取指侧的直接跳转静态预测和返回地址预测均通过参数控制。默认配置优先保持 IF 的 PC 反馈路径简洁；关闭这些可选预测不会影响指令正确性，因为所有控制转移最终都在 EX2 核对并恢复。

## 9.3 当前功能边界

从现有 RTL 可见，设计已经覆盖整程序执行、乘除法、动态分支预测、同步存储访问和基本机器模式系统调用。当前异常入口主要围绕 `ecall` 构建，尚未形成外部中断、非法指令、地址未对齐和访问故障的统一异常仲裁。载入存储接口也默认软件遵守自然对齐约束。上述边界不影响本文三组功能验证，但在扩展为完整机器模式处理器时需要继续完善。

# 10 总结与展望

本文依据实际 RTL 对 KLDJ 流水线 CPU 进行了系统说明。处理器使用 IF、ID、EX、MEM1、MEM2、WB 六级主数据通路，控制转移通过独立 EX2 边界完成方向和目标核对。译码与前递选择在较早阶段生成，EX 支持整数、分支、地址生成和多周期乘除法；同步载入响应在 MEM2 前完成格式化，使紧邻 load-use 相关只需一个停顿周期。前端的 gshare PHT、BTB 和返回地址栈为控制流加速提供基础，统一重定向和逐级有效位则保证预测不会改变程序语义。

在系统侧，真双端口 BRAM 分离读写路径，单项写缓冲解决注册写路径带来的写后读问题；CSR 文件与 EX2 重定向共同实现 `ecall` 入口和 `mret` 返回。三组功能仿真从存储一致性、预测状态机和系统控制闭环三个角度验证关键设计。

后续工作可从三个方向展开：一是加入统一的中断和异常仲裁，完善 `mtvec` 模式、地址未对齐及访问故障处理；二是增加提交级参考模型对比和随机指令测试，提高验证覆盖率；三是基于真实应用统计分支命中率、停顿来源和 IPC，为预测表规模、乘除法结构及存储流水深度的取舍提供量化依据。

## A 主要 RTL 文件结构与职责对应表

本项目采用“系统集成层–CPU 核心顶层–阶段模块/流水控制–运算与通用单元”的分层组织方式。目录树如清单 1 所示，仅展开本文涉及的基础 CPU 实现和系统集成文件。

Listing 1 项目 RTL 目录层次

```

pipilined5/
|-- kldj.srscs2/
|   |-- sources_1/imports/rtl/
|   |   |-- KLDJ_top.v
|   |   |-- KLDJ_perf_counters.v
|   |   |-- define.v
|   |   |-- stage/
|   |       |-- KLDJ_ifu.v      KLDJ_idu.v      KLDJ_exu.v
|   |       |-- KLDJ_lsu.v     KLDJ_wbu.v
|   |       |-- KLDJ_regfile.v KLDJ_csr.v
|   |   |-- pipe/
|   |       |-- pipe_if_id.v    pipe_id_ex.v    pipe_ex_mem.v
|   |       |-- pipe_mem_wb.v   pipe_wb_commit.v
|   |       |-- ex_forward.v    ex_mem_req_ctrl.v mem_stage_top.v
|   |       |-- bpu.v          ex_bpu_ctrl.v
|   |   |-- alu/
|   |       |-- KLDJ_alu.v      alu_add.v      alu_and.v
|   |       |-- alu_or.v       alu_xor.v      alu_shifter.v
|   |       |-- alu_slt.v      mul_ip_wrapper.v div_ip_wrapper.v
|   |       |-- alu_mul.v      alu_div.v
|   |   |-- util/
|   |       |-- DataMemory.v
|   |       |-- MuxKeyInternal.v
|   |       |-- MuxKeyWithDefault.v
|-- new/
|   |-- top.sv          student_top.sv      IROM_BHV.sv
|   |-- perip_bridge.sv dram_driver.sv      DRAM_TDP.sv
|   |-- counter.sv      display_seg.sv

```

从逻辑实例关系看，板级 top 通过 student\_top 连接指令存储器、KLDJ\_top 和 perip\_bridge；KLDJ\_top 再统一例化各流水阶段、预测控制和流水寄存器。执行级 KLDJ\_exu 向下调用 ALU 与乘除法包装器，而数据侧经 perip\_bridge、dram\_driver 连接双端口 BRAM。表 A.1 按此层次说明各目录中的实际文件职责。

表 A.1 RTL 文件层次与职责对应表

| 目录层级                                           | 实际文件/模块                                         | 主要职责                                                        |
|------------------------------------------------|-------------------------------------------------|-------------------------------------------------------------|
| <b>第 1 层：CPU 核心顶层 (sources_1/imports/rtl/)</b> |                                                 |                                                             |
| 核心顶层                                           | KLDJ_top.v                                      | 连接取指、译码、执行、访存、写回、分支预测和 CSR；生成全局停顿、冲刷和 EX2 控制结果边界。           |
| 基础定义                                           | define.v                                        | 定义数据宽度、起始 PC、指令主操作码、内部操作码和 CSR 地址等全局宏。                      |
| 观察单元                                           | KLDJ_perf_counters.v                            | 定义周期、提交、停顿、重定向和访存事件的独立计数器，作为后续性能观察接口。                       |
| <b>第 2 层：流水阶段模块 (rtl/stage/)</b>               |                                                 |                                                             |
| 取指阶段                                           | KLDJ_ifu.v                                      | 保存 PC，选择顺序地址、预测地址或重定向地址，并响应前端保持控制。                          |
| 译码阶段                                           | KLDJ_idu.v                                      | 解析指令字段，生成立即数、寄存器读写控制、ALU/访存/CSR 控制与四路操作数。                   |
| 执行阶段                                           | KLDJ_exu.v                                      | 完成整数运算、分支比较、跳转目标、访存地址、乘除法状态以及 CSR 写数据生成。                    |
| 访存与写回                                          | KLDJ_lsu.v,<br>KLDJ_wbu.v                       | KLDJ_lsu 选择载入字节/半字并扩展；KLDJ_wbu 将最终结果送往写回端。                  |
| 寄存器堆                                           | KLDJ_regfile.v                                  | 实现 32 个通用寄存器、双读单写接口、x0 保护和同周期 write-through 旁路。             |
| 机器模式状态                                         | KLDJ_csr.v                                      | 实现 mstatus、mtvec、mepc、mcause、mscratch，并处理 ecall/mret 的状态更新。 |
| <b>第 3 层：流水寄存器与控制单元 (rtl/pipe/)</b>            |                                                 |                                                             |
| 阶段寄存器                                          | pipe_if_id.v,<br>pipe_id_ex.v,<br>pipe_ex_mem.v | 在 IF/ID、ID/EX、EX/MEM 边界寄存指令、PC、操作数、控制信号、有效位与前递选择。           |

| 目录层级      | 实际文件/模块                            | 主要职责                                                             |
|-----------|------------------------------------|------------------------------------------------------------------|
| 访存/提交边界   | pipe_mem_wb.v,<br>pipe_wb_commit.v | 前者同时定义 MEM1/MEM2 响应寄存器和 MEM/WB 寄存器；后者提供提交观察点。                    |
| 数据相关控制    | ex_forward.v,<br>ex_mem_req_ctrl.v | 预计算并选择 EX 前递来源，检测 load-use 停顿；生成数据地址、写数据、写使能和字节使能。               |
| MEM2 数据组织 | mem_stage_top.v                    | 选择已寄存的载入结果或执行结果，形成与前递一致的写回数据。                                    |
| 分支预测单元    | bpu.v                              | 包含 gshare_btb_core 与 return_address_stack，实现 PHT、BTB、GHR 和返回地址栈。 |
| 控制转移核对    | ex_bpu_ctrl.v                      | 比较预测方向/目标与实际结果，产生重定向，并更新 BPU 与返回地址栈状态。                           |

#### 第 4 层：运算与通用单元 (rtl/alu/, rtl/util/)

|         |                                                                              |                                       |
|---------|------------------------------------------------------------------------------|---------------------------------------|
| ALU 顶层  | KLDJ_alu.v                                                                   | 汇聚加减比较、逻辑、移位与小于比较子单元，输出运算结果和分支比较标志。   |
| 基础运算子单元 | alu_add.v, alu_and.v,<br>alu_or.v, alu_xor.v,<br>alu_shifter.v,<br>alu_slt.v | 分别实现加减/比较、逻辑、移位和置位功能，供 KLDJ_alu 组合调用。 |
| 长周期运算   | mul_ip_wrapper.v,<br>div_ip_wrapper.v                                        | 包装乘法和除法 IP 的启动、忙、完成和结果接口，并处理符号与边界条件。  |
| 兼容运算文件  | alu_mul.v, alu_div.v                                                         | 保留兼容模块接口；实际乘除法数据通路由对应的 IP 包装器实现。      |
| 通用工具    | DataMemory.v,<br>MuxKeyInternal.v,<br>MuxKeyWithDefault.v                    | 提供基础存储器模型和可复用键值选择器，便于局部模块或测试环境复用。     |

#### 第 5 层：系统集成与存储器层 (pipilined5/new/)

| 目录层级        | 实际文件/模块                                           | 主要职责                                                            |
|-------------|---------------------------------------------------|-----------------------------------------------------------------|
| 板级封装        | top.sv,<br>student_top.sv                         | 连接时钟、复位、IROM、CPU 核心、<br>外设桥和板级输入输出；<br>student_top 例化 KLDJ_top。 |
| 指令存储器       | IROM_BHV.sv                                       | 提供与指令存储 IP 兼容的行为级模<br>型，用于仿真读取指令。                               |
| 地址译码与外<br>设 | perip_bridge.sv,<br>counter.sv,<br>display_seg.sv | 进行 DRAM/MMIO 地址译码，连接<br>开关、按键、LED、数码管和计数器<br>等外设。               |
| 数据 BRAM     | dram_driver.sv,<br>DRAM_TDP.sv                    | 以真双端口 BRAM 提供读写通路；<br>dram_driver 负责注册写路径和逐字<br>节写后读前递。         |

## 参考文献

- [1] RISC-V International. *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*.
- [2] RISC-V International. *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture*.
- [3] AMD Xilinx. *7 Series FPGAs Memory Resources User Guide (UG473)*.
- [4] David A. Patterson, John L. Hennessy. *Computer Organization and Design RISC-V Edition*. Morgan Kaufmann.